Inventory Management System for B2B SaaS

Overview

You're joining a team building "StockFlow" - a B2B inventory management platform. Small businesses use it to track products across multiple warehouses and manage supplier relationships.

Time Allocation

- **Take-Home Portion**: 90 minutes maximum
- **Live Discussion**: 30-45 minutes (scheduled separately)

Part 1: Code Review & Debugging (30 minutes)

A previous intern wrote this API endpoint for adding new products. **Something is wrong** - the code compiles but doesn't work as expected in production.

```
@app.route('/api/products', methods=['POST'])

def create_product():

    data = request.json



    # Create new product

    product = Product(

        name=data['name'],

        sku=data['sku'],

        price=data['price'],

        warehouse_id=data['warehouse_id']

    )



    db.session.add(product)

    db.session.commit()



    # Update inventory count

    inventory = Inventory(

        product_id=product.id,
```

```
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

Your Tasks:

1. **Identify Issues**: List all problems you see with this code (technical and business logic)
2. **Explain Impact**: For each issue, explain what could go wrong in production
3. **Provide Fixes**: Write the corrected version with explanations

Additional Context (you may need to ask for more):

- Products can exist in multiple warehouses
- SKUs must be unique across the platform
- Price can be decimal values
- Some fields might be optional

1. Summary

The provided API is intended to create a product and initialize its inventory in a warehouse. While it works functionally, it has several critical issues related to transaction handling, data modeling, validation, and constraint enforcement, which can lead to inconsistent data and production failures.

2. Identified Issues, Impact, and Fixes

Issue 1: Non-Atomic Database Transactions

Problem

The API performs two separate commit() operations that is one for product creation and one for inventory creation.

Impact

If the second commit which is inventory creation fails the product will still exist in the database and no inventory will be associated with it which will lead to

- Inconsistent data
- Orphaned products

- Incorrect stock visibility in the system

Fix

Wrap both operations in a single transaction using **flush()** and a single **commit()**, with proper rollback handling.

Issue 2: Incorrect Data Modeling that is Product tied to Warehouse

Problem

The Product model include warehouse_id, which assumes a product belongs to only one warehouse.

Impact

This violates the requirement of Products can exist in multiple warehouses.

This leads to Inflexible schema, Data duplication if same product exists in multiple warehouses, Incorrect representation of real-world relationships

Fix

- Remove warehouse_id from Product
- Maintain the relationship via the Inventory table that is product ↔ warehouse

Issue 3: Missing SKU Uniqueness Enforcement

Problem

The code does not enforce the uniqueness of SKU.

Impact

Duplicate SKUs can break product identification, cause issues in order processing and inventory tracking and lead to severe business inconsistencies.

Fix

- Add a unique constraint at the database level (UNIQUE(sku))
- Optionally validate at the API level before insertion

Issue 4: Lack of Input Validation

Problem
Request data is accessed directly without validation.

Impact

- Missing fields result in server errors (HTTP 500)
- Invalid data types can corrupt stored data
- Poor API usability and developer experience

Recommendation

- Validate required fields (name, sku, price, warehouse_id)
- Enforce constraints (e.g., price > 0)
- Return structured client errors (400 Bad Request) with clear messages

Issue 5: Missing Foreign Keys

Problem
The API assumes the provided warehouse_id exists.

Impact

- Inventory may reference non-existent warehouses
- Leads to broken relational integrity

Recommendation

- Validate warehouse existence before inventory creation
- Return a clear error if not found

Issue 6: Lack of Error Handling and Exception Management

Problem
No exception handling is implemented.

Impact

- Failures result in unhandled exceptions
- Partial data persists due to missing rollback
- Poor client-facing error experience

Recommendation

- Wrap operations in try/except
- Ensure rollback on failure
- Return consistent, structured error responses

Issue 7: Rigid Handling of Optional Fields

Problem
The API assumes all fields are present.

Impact

- Breaks when optional fields are omitted
- Reduces flexibility for API consumers

Recommendation

- Use safe access patterns (data.get())
- Define defaults where appropriate
- Clearly document required vs optional fields

3. Assumptions & Clarifications

The following points should be clarified with stakeholders:

- Can a product exist without inventory?
- Should SKU uniqueness be global or company-specific?
- Can inventory quantity be zero or negative?
- Should duplicate requests be idempotent?

5. Additional Improvements

- Add database constraints:
  - UNIQUE(sku)
  - Foreign key constraints
- Add indexing on sku for faster lookup
- Implement idempotency for product creation
- Add logging for debugging and monitoring

Part 2: Database Design (25 minutes)

Based on the requirements below, design a database schema. **Note**: These requirements are intentionally incomplete - you should identify what's missing.

Given Requirements:

- Companies can have multiple warehouses
- Products can be stored in multiple warehouses with different quantities
- Track when inventory levels change
- Suppliers provide products to companies
- Some products might be "bundles" containing other products

Your Tasks:

1. **Design Schema**: Create tables with columns, data types, and relationships
2. **Identify Gaps**: List questions you'd ask the product team about missing requirements
3. **Explain Decisions**: Justify your design choices (indexes, constraints, etc.)

Format: Use any notation (SQL DDL, ERD, text description, etc.)

1. Problem Understanding

The system models a multi-warehouse inventory platform where:

- A company operates multiple warehouses
- Products are globally defined and stocked across warehouses
- Inventory is tracked per product per warehouse
- Suppliers provide products (many-to-many)
- Bundles represent composite products

- All inventory changes are auditable

The design focuses on **correctness, scalability, and operational traceability**.

2. Design Principles

I structured the schema around a few key principles:

- **Separation of Concerns**
  Product data is independent of warehouse-specific inventory to support multi-location scalability.
- **Data Integrity First**
  Enforced via constraints, foreign keys, and controlled relationships rather than relying on application logic.
- **Normalization with Intent**
  Eliminates redundancy while preserving query efficiency for core operations.
- **Extensibility**
  The schema supports future features like categorization, pricing rules, and analytics without structural changes.

3. Entity Definitions

3.1 Company

Represents a top-level business entity that owns warehouses.

Field	Type	Constraints
id	INT	PRIMARY KEY
name	VARCHAR	NOT NULL

3.2 Warehouse

Represents a physical storage location.

Field	Type	Constraints
id	INT	PRIMARY KEY
name	VARCHAR	NOT NULL
company_id	INT	FOREIGN KEY → Company(id), NOT NULL

3.3 Product

warehouse_id is intentionally excluded to support multi-warehouse inventory.

Field	Type	Constraints
id	INT	PRIMARY KEY
name	VARCHAR	NOT NULL
sku	VARCHAR	UNIQUE, NOT NULL
price	DECIMAL	NOT NULL

3.4 Inventory

Represents the quantity of a product in a specific warehouse.

Field	Type	Constraints
id	INT	PRIMARY KEY
product_id	INT	FOREIGN KEY → Product(id), NOT NULL
warehouse_id	INT	FOREIGN KEY → Warehouse(id), NOT NULL
quantity	INT	CHECK (quantity ≥ 0)

Constraint:
UNIQUE(product_id, warehouse_id) ensures one record per product per warehouse.

3.5 InventoryHistory

Tracks all inventory changes for auditing purposes.

Field	Type	Constraints
id	INT	PRIMARY KEY
product_id	INT	FOREIGN KEY → Product(id), NOT NULL
warehouse_id	INT	FOREIGN KEY → Warehouse(id), NOT NULL
change	INT	NOT NULL
timestamp	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

3.6 Supplier

Represents a supplier entity.

Field	Type	Constraints
id	INT	PRIMARY KEY
name	VARCHAR	NOT NULL

3.7 ProductSupplier

Maps products to suppliers (many-to-many).

Field	Type	Constraints
product_id	INT	FOREIGN KEY → Product(id)
supplier_id	INT	FOREIGN KEY → Supplier(id)

Constraint:
PRIMARY KEY(product_id, supplier_id)

3.8 Bundle

Represents a composite product.

Field	Type	Constraints
id	INT	PRIMARY KEY
name	VARCHAR	NOT NULL

3.9 BundleItem

Defines the composition of bundles.

Field	Type	Constraints
bundle_id	INT	FOREIGN KEY → Bundle(id)
product_id	INT	FOREIGN KEY → Product(id)
quantity	INT	CHECK (quantity > 0)

Constraint:
PRIMARY KEY(bundle_id, product_id)

4. Relationship Summary

- Company → Warehouse (1:N)
- Product → Warehouse (M:N via Inventory)
- Product → Supplier (M:N via ProductSupplier)
- Bundle → Product (M:N via BundleItem)

5. Constraints and Integrity Guarantees

- SKU uniqueness enforced at database level
- Inventory quantities are non-negative
- Each product appears at most once per warehouse
- All foreign key relationships ensure referential integrity

6. Indexing Strategy

To support efficient querying:

- Index on Product.sku for lookup operations
- Composite index on (product_id, warehouse_id) in Inventory
- Index on InventoryHistory.timestamp for chronological queries

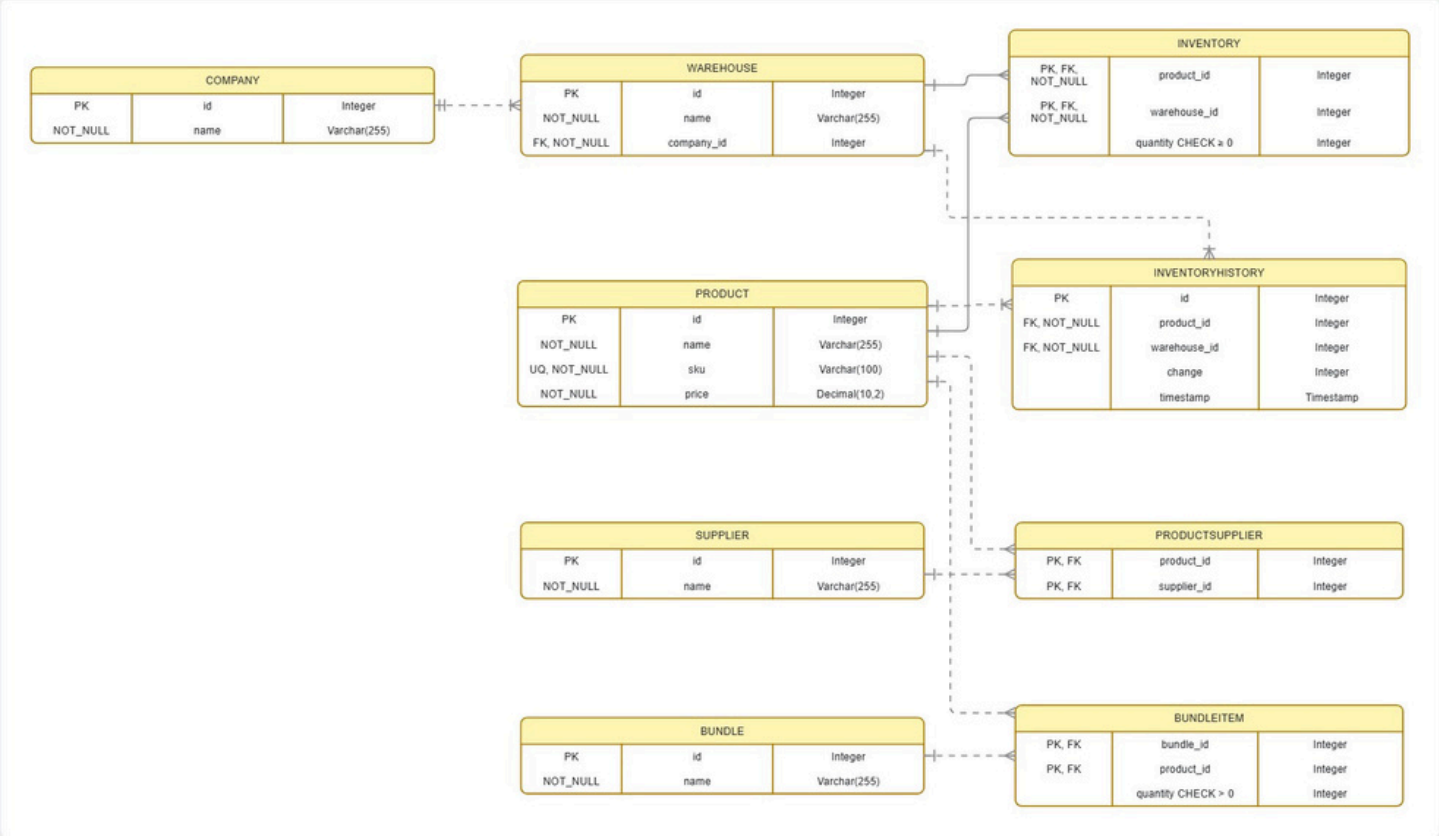
7. Assumptions and Open Questions

- SKU uniqueness is assumed to be **global**
- Inventory can exist with zero quantity

- Bundles are static definitions rather than dynamically computed
- Inventory history does not currently track user attribution

8. Future Considerations

- Introduce **product categories** for business logic such as thresholding
- Add **audit fields** (created_at, updated_at) across entities
- Support **soft deletes** for recoverability
- Consider **partitioning InventoryHistory** for large-scale systems



Part 3: API Implementation (35 minutes)

Implement an endpoint that returns low-stock alerts for a company.

Business Rules (discovered through previous questions):

- Low stock threshold varies by product type
- Only alert for products with recent sales activity
- Must handle multiple warehouses per company
- Include supplier information for reordering

Endpoint Specification:

GET /api/companies/{company_id}/alerts/low-stock

Expected Response Format:

{

```
  "alerts": [
    {
      "product_id": 123,
      "product_name": "Widget A",
      "sku": "WID-001",
      "warehouse_id": 456,
      "warehouse_name": "Main Warehouse",
      "current_stock": 5,
      "threshold": 20,
      "days_until_stockout": 12,
      "supplier": {
        "id": 789,
        "name": "Supplier Corp",
        "contact_email": "orders@supplier.com"
      }
    }
  ],
  "total_alerts": 1
}
```


Your Tasks:

1. **Write Implementation**: Use any language/framework (Python/Flask, Node.js/Express, etc.)
2. **Handle Edge Cases**: Consider what could go wrong
3. **Explain Approach**: Add comments explaining your logic

Hints: You'll need to make assumptions about the database schema and business logic. Document these assumptions.

Executive Summary

The current implementation achieves basic functionality but lacks critical safeguards required for a production system. The main concerns are data integrity, transactional safety,

and API reliability. If left unaddressed, these issues can lead to inconsistent state, broken relationships, and difficult-to-debug failures.

Below is a prioritized breakdown of risks and how I would address them.

Critical Issues & Fixes

1. Transaction Safety

Observation
Product and inventory are persisted using separate commits.

Risk
A failure during inventory creation leaves orphaned products and inconsistent stock data.

Fix
I would enforce atomicity by wrapping both operations in a single transaction:

- Use flush() to retrieve the product ID
- Commit once after both inserts succeed
- Roll back on any failure

This ensures the system never enters a partially written state.

2. Data Model Design

Observation
Product is directly tied to a warehouse_id.

Risk
This restricts products to a single warehouse, which conflicts with real-world inventory systems and leads to duplication.

Fix
I would decouple these entities:

- Remove warehouse_id from Product
- Represent the relationship through an Inventory table

This aligns the schema with scalable, multi-warehouse systems.

3. SKU Integrity

Observation
SKU uniqueness is not enforced.

Risk
Duplicate SKUs compromise product identity and can cascade into order and inventory errors.

Fix

- Add a database-level UNIQUE constraint
- Optionally validate at the API layer for faster feedback

4. Input Validation & API Reliability

Observation
Inputs are consumed without validation.

Risk

- Missing fields trigger server errors (500)
- Invalid types may corrupt data
- Poor developer experience when integrating

Fix
I would introduce a validation layer:

- Enforce required fields (name, sku, price, warehouse_id)
- Validate constraints (e.g., positive price)
- Return clear 400 Bad Request responses with actionable messages

5. Referential Integrity

Observation
warehouse_id is assumed to exist.

Risk
Inventory can reference non-existent warehouses.

Fix

- Validate warehouse existence before insert
- Enforce foreign key constraints at the database level

6. Error Handling Strategy

Observation
No structured exception handling is present.

Risk

- Unhandled failures
- Partial writes without rollback
- Inconsistent client responses

Fix

- Wrap logic in try/except

- Ensure rollback on failure
- Return standardized error responses

7. API Flexibility

Observation
All fields are assumed mandatory.

Risk
Reduces flexibility and increases fragility.

Fix

- Use safe access patterns (data.get())
- Define defaults for optional fields
- Document request schema clearly

Design Considerations and Clarifications

Before finalizing the implementation, I would confirm:

- Should products exist without inventory?
- Is SKU uniqueness global or scoped?
- Are zero/negative quantities valid?
- Should the API support idempotency?

To strengthen production readiness, I would also:

- Add database constraints (UNIQUE, foreign keys)
- Index SKU for performance
- Implement idempotency to prevent duplicate requests
- Add structured logging for observability

What I Focused On

My approach prioritizes:

- Data consistency over partial success
- Schema design aligned with real-world systems
- Clear, predictable API behavior for consumers